

Structured Reasoning Traces and Parse Tree Hierarchies for Grounding Arithmetic in Language Models

Deepanshi Mohit Sinsniwal
singladeepanshi6@gmail.com mohit.sinsniwal@gmail.com

Advisor: Prof. K.V. Subramanyam

GitHub Repository:
<https://github.com/mira-iitjmu/math-grounding-IR>

Contents

1	Abstract	3
2	Introduction	3
3	Parse Trees as Intermediate Representations	5
3.1	Why a Single Tree is Not Enough	5
3.2	The Four-Level IR Hierarchy	6
3.2.1	Stage 1: Raw AST	6
3.2.2	Stage 2: Logic AST	7
3.2.3	Stage 3: Canonical AST	7
3.2.4	Stage 4: Abstract AST	8
3.3	Purpose of Each Representation	8
4	The Reasoning Trace Grammar	8
4.1	Why Parse Trees Alone are Insufficient	8
4.2	Formal AST Grammar (GAST)	9
4.3	Reasoning Trace Grammar (GRT)	9
4.4	Example Walkthrough: Meena’s Pencils	10
4.4.1	Entity Extraction	10
4.4.2	Concept Mapping	10
4.4.3	Variable Binding	10
4.4.4	Final Construction	10
4.5	Disambiguation Rules	11
4.6	Verification Checks	12

5	System Architecture and Implementation	12
5.1	Forward Pipeline	12
5.1.1	Stage 1: LLM-Based Trace Generation	13
5.1.2	Stage 2: Parsing Layer	13
5.1.3	Stage 3: Schema Validation	13
5.1.4	Stage 4: Symbolic Execution	13
5.1.5	Stage 5: Verification Checks	14
5.1.6	Stage 6: IR Conversion and Storage	14
5.2	Reverse Pipeline	14
5.3	Equivalence Simulator	14
5.4	Workspace Infrastructure	15
6	Dataset Generation and Fine-Tuning Pipeline	15
6.1	Structural Clustering and Operation Balancing	15
6.2	Vocabulary Augmentation	15
6.3	Training Schema	16
6.4	Curriculum Learning	16
6.5	Programmatic Verification	16
7	Results and Observations	16
7.1	Model Size and JSON Extraction Reliability	16
7.2	Equivalence Simulator Results	17
7.3	Batch Evaluation	18
8	Limitations and Future Work	18
8.1	Current Limitations	18
8.2	Future Work	19
9	Conclusion	20
A	LLM Prompt Templates	20
A.1	Reasoning Trace Generation Prompt	21
A.2	Raw AST Extraction Prompt (Forward Pipeline)	22
A.3	Reverse Pipeline Prompt — Abstract to Canonical	22
A.4	Reverse Pipeline Prompt — Canonical to Raw	22
A.5	Reverse Pipeline Prompt — Raw to Word Problem	23
B	References	23

...

1 Abstract

Large language models produce arithmetic solutions as unstructured natural language, making intermediate reasoning steps difficult to verify programmatically. This project introduces a dual-grammar verification framework that separates linguistic understanding from mathematical computation.

A formal Reasoning Trace Grammar (GRT) decomposes word problems into explicit cognitive steps including entity extraction, semantic mapping, operand binding, and tree construction. The generated trace compiles into a binary Abstract Syntax Tree governed by a second grammar called GAST.

The compiled tree is processed through a four-level intermediate representation hierarchy consisting of Raw, Logic, Canonical, and Abstract forms. This hierarchy enables symbolic execution, deterministic deduplication, structural clustering, and synthetic problem generation.

The framework was evaluated on NCERT Class 3–5 arithmetic word problems and demonstrated the ability to map structurally identical problems into identical canonical representations irrespective of surface wording.

2 Introduction

Consider a student who reads the problem:

“Meena had 180 pencils. She gave 95 pencils to her friends and later bought 60 more.”

The student writes:

$$180 + 95 + 60 = 335$$

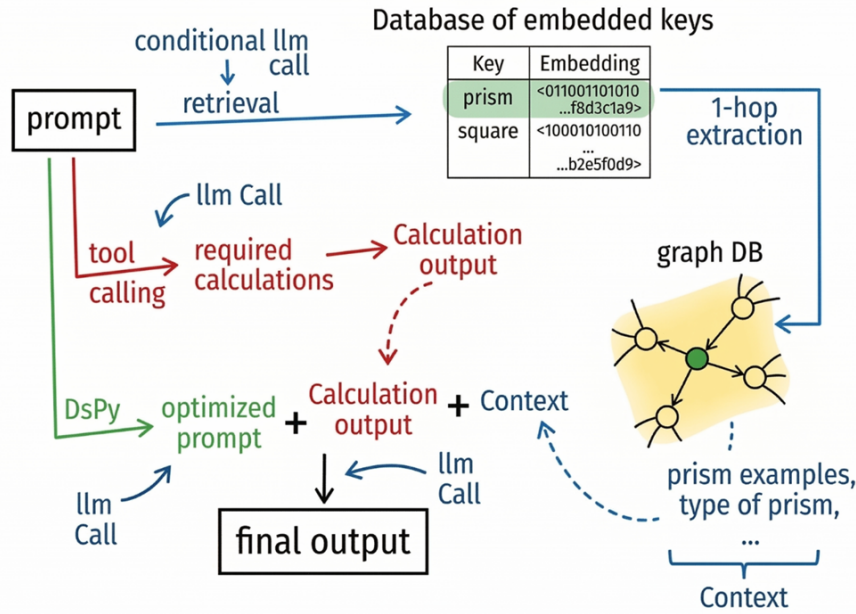
The arithmetic is executed correctly, but the answer is wrong. The failure is not computational it is semantic. The student failed to recognise that “gave” implies subtraction, not addition. The correct formulation is:

$$ADD(SUB(180, 95), 60) = 145$$

Language models exhibit an analogous failure mode, but in the opposite direction. A model can produce a fluent chain-of-thought trace complete with plausible-sounding intermediate steps while computing incorrect values at one or more stages. The output reads well, but the math is wrong.

Three specific failure modes are common when LLMs are used for arithmetic word problems:

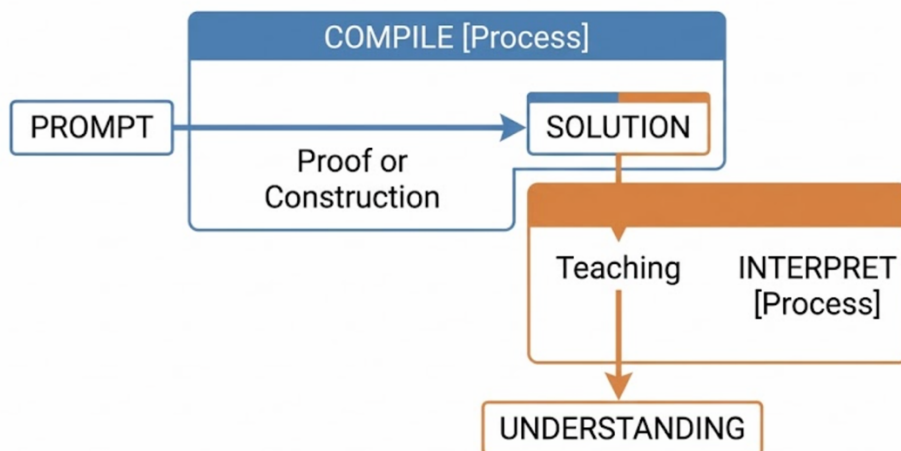
1. Incomplete handling of input quantities
2. Incorrect intermediate calculations embedded within otherwise coherent text
3. Deviation from standard textbook solution methodology



The root cause is a lack of grounding. The model's output is free-form text, and there is no formal structure to parse, execute, or validate. When the output is wrong, there is no way to isolate which step failed the entire generation must be discarded and re-attempted.

This problem is compounded in educational contexts, where the goal is not merely to produce a correct answer but to support learning. Solving and teaching are fundamentally different tasks. Solving optimises for the best answer. Teaching optimises for understanding what the student lacks and helping them reach that understanding on their own.

A tutoring system that simply reveals the answer eliminates the productive struggle that builds genuine comprehension, while a system that gives generic hints cannot respond to the specific error a student has made.



Both applications reliable solving and structured teaching require the same underlying capability: a structured, verifiable representation of the solution path.

The representation must satisfy four properties:

1. It must be computable
2. It must be verifiable
3. It must be storable
4. It must be reusable

This project introduces a dual-grammar framework that decouples semantic text parsing from mathematical calculation.

The framework operates in two layers:

- A Reasoning Trace Grammar (GRT) that governs linguistic decomposition of word problems into discrete cognitive steps
- An Abstract Syntax Tree Grammar (GAST) that governs mathematical execution structure

The output of GRT compiles directly into a GAST tree, which is then processed through a four-level intermediate representation hierarchy to enable validation, clustering, and synthetic problem generation.

The current implementation targets NCERT Class 3–5 arithmetic word problems involving addition, subtraction, multiplication, and division as a proof of concept.

3 Parse Trees as Intermediate Representations

3.1 Why a Single Tree is Not Enough

A natural way to represent arithmetic reasoning is through a binary Abstract Syntax Tree (AST). Consider the following problem:

“A photo frame is 45 cm long and 30 cm wide. Find the perimeter of the frame.”

The perimeter formula is represented as:

$$MUL(2, ADD(45, 30))$$

Although this representation is mathematically correct, it combines two different types of information:

- Problem-specific numeric values
- General operator structure

The operator structure remains identical across all rectangle perimeter problems, regardless of the dimensions used. However, a single AST representation cannot distinguish between reusable reasoning structure and instance-specific values.

Another challenge arises from commutativity. The expressions:

$$ADD(45, 30)$$

and

$$ADD(30, 45)$$

are mathematically equivalent but produce different string representations. This creates problems for hashing, deduplication, and retrieval systems.

To solve these issues, the framework introduces a four-level Intermediate Representation (IR) hierarchy.

3.2 The Four-Level IR Hierarchy

The system processes each parse tree through four abstraction stages.

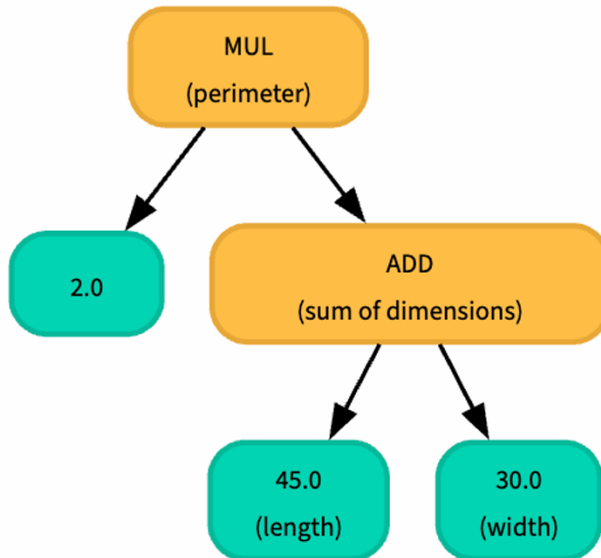
3.2.1 Stage 1: Raw AST

The Raw AST preserves contextual labels extracted from the original word problem.

Example:

$$MUL[perimeter](2.0, ADD[sum](45.0[length], 30.0[width]))$$

Matches exact problem layout and keeps textual labels.



This representation retains semantic grounding information.

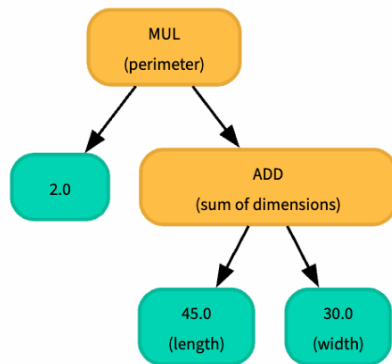
3.2.2 Stage 2: Logic AST

The Logic AST removes contextual labels while preserving the mathematical structure.
Example:

$$MUL(2.0, ADD(45.0, 30.0))$$

1. Raw

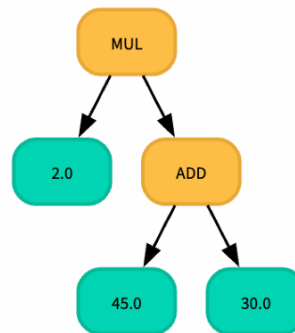
Matches exact problem layout and keeps textual labels.



> Show JSON

2. Logic

Stripped of contextual text labels. Pure math structure.



> Show JSON

The Logic AST is symbolically executed to verify arithmetic correctness.

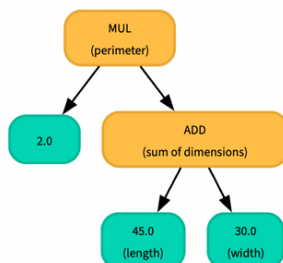
3.2.3 Stage 3: Canonical AST

The Canonical AST sorts operands of commutative operators deterministically.
Example:

$$MUL(2.0, ADD(30.0, 45.0))$$

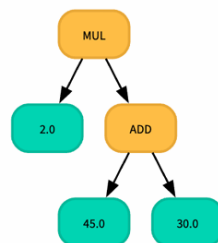
1. Raw

Matches exact problem layout and keeps textual labels.



2. Logic

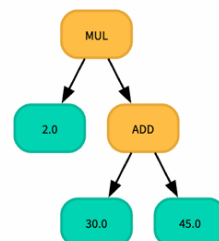
Stripped of contextual text labels. Pure math structure.



> Show JSON

3. Canonical

Commutative operations structurally sorted for deterministic hashing.



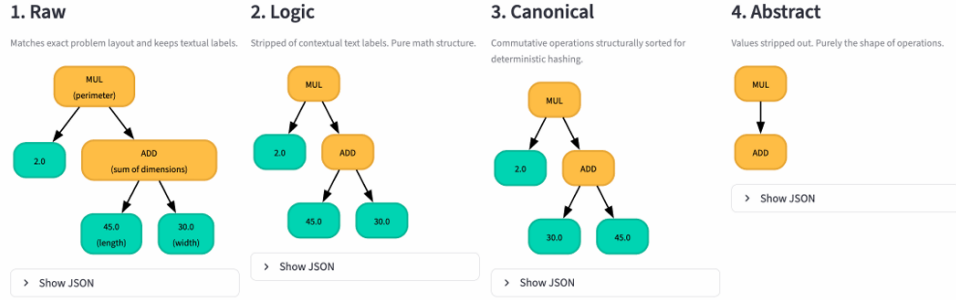
This normalization ensures that mathematically equivalent expressions produce identical hashes.

3.2.4 Stage 4: Abstract AST

The Abstract AST masks all scalar values while preserving only the reasoning structure.

Example:

$$MUL(val, ADD(val, val))$$



This representation captures the abstract reasoning pattern independent of numbers or story context.

3.3 Purpose of Each Representation

Each representation level serves a different role:

- Logic AST enables symbolic execution and verification
- Canonical AST supports hashing and deduplication
- Abstract AST enables structural clustering and synthetic generation

The conversion pipeline is fully deterministic after the initial language model extraction stage.

4 The Reasoning Trace Grammar

4.1 Why Parse Trees Alone are Insufficient

The four-level IR hierarchy captures the mathematical structure of a solution, but it does not capture the reasoning process that transforms natural language into mathematical operations.

For example, the sentence:

“She gave 95 pencils”

requires understanding that the word “gave” implies subtraction.

Similarly:

“A debt of 20 dollars”

represents a negative quantity rather than a subtraction operation.

Although both situations involve negative semantics, their structural interpretations are different. A parse tree stores only the final structure and not the reasoning process used to derive it.

To capture explicit reasoning steps, the framework introduces the Reasoning Trace Grammar (GRT).

4.2 Formal AST Grammar (GAST)

The mathematical execution layer is governed by a strict context-free grammar written in Extended Backus-Naur Form (EBNF).

```
ast_root      ::= node
node          ::= binary_operation | unary_operation | terminal

binary_operation ::= binary_operator [ metadata ] "(" node "," node ")"
unary_operation  ::= unary_operator [ metadata ] "(" node ")"

binary_operator ::= "ADD" | "SUB" | "MUL" | "DIV" | "POW" | "MOD" | "MIN" | "MAX"
unary_operator  ::= "NEG" | "ABS" | "SQRT" | "LOG"

terminal       ::= literal | variable
literal        ::= [ "-" ] digit { digit } [ "." digit { digit } ]
variable       ::= identifier [ metadata ]
metadata       ::= "[" string_literal "]"
```

Every binary operator contains exactly two children, while unary operators contain exactly one child.

4.3 Reasoning Trace Grammar (GRT)

The Reasoning Trace Grammar represents the reasoning process as a Directed Acyclic Graph (DAG).

Each trace step performs one cognitive action:

- Entity extraction
- Semantic mapping
- Operand binding
- Final tree construction

The reasoning trace follows the structure:

```

reasoning_trace    ::= { trace_step } final_construction
trace_step        ::= step_id [dependency_list] step_body "\n"

step_id           ::= "[" identifier "]"
dependency_list   ::= "<" identifier { "," identifier } ">"

extraction_body   ::= "EXTRACT" "entity:" string_literal "->" "val:" terminal
mapping_body      ::= "MAP" "trigger:" string_literal "->" "gen_trigger:" string_literal
                    "->" "concept:" string_literal "->" "op:" operator
binding_body      ::= "BIND" "logic:" string_literal "->" "role:" role_assignment

role_assignment   ::= "LEFT" "=" node | "RIGHT" "=" node | "ARG" "=" node
final_construction ::= "[CONSTRUCT]" "Tree:" ast_root [ "->" "target:" string_literal

```

Dependencies between steps explicitly define the DAG structure.

4.4 Example Walkthrough: Meena's Pencils

Consider the problem:

“Meena had 180 pencils. She gave 95 pencils to her friends and later bought 60 more.”

4.4.1 Entity Extraction

```

[E1] EXTRACT entity: "180 pencils" -> val: 180
[E2] EXTRACT entity: "95 pencils" -> val: 95
[E3] EXTRACT entity: "60 more" -> val: 60

```

4.4.2 Concept Mapping

```

[M1] MAP trigger: "gave" -> gen_trigger: "give" -> concept: "decrease" -> op: SUB
[M2] MAP trigger: "bought" -> gen_trigger: "buy" -> concept: "increase" -> op: ADD

```

4.4.3 Variable Binding

```

[B1] <E1, E2, M1> BIND logic: "Initial pencils minus pencils given away" -> role: LEFT
[B2] <B1, E3, M2> BIND logic: "Current pencils plus pencils bought" -> role: LEFT

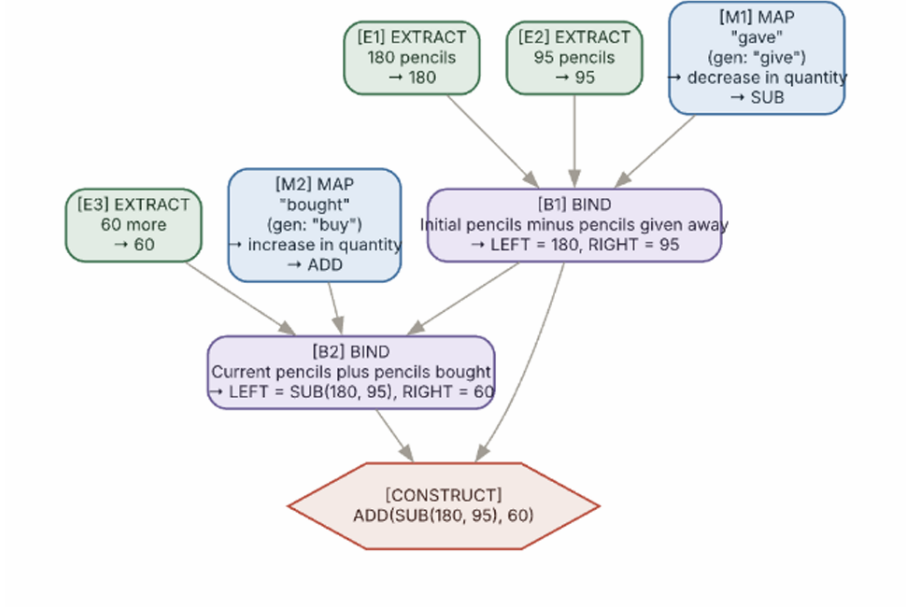
```

4.4.4 Final Construction

```

[CONSTRUCT] Tree: ADD(SUB(180, 95), 60) -> target: "pencils"

```



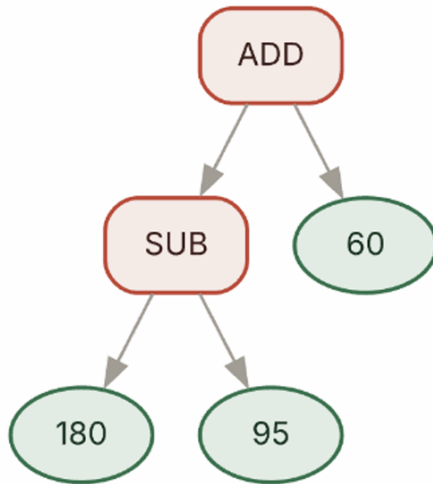
The final AST is symbolically executed:

$$SUB(180, 95) = 85$$

$$ADD(85, 60) = 145$$

Thus, the final answer is:

145



4.5 Disambiguation Rules

The framework separates three different meanings of negative semantics.

1. Binary subtraction:

$$SUB(State, 20)$$

2. Negative literals:

$$-20$$

3. Unary negation:

$$NEG(X)$$

This separation prevents ambiguity during parsing and execution.

Natural Language Trigger	Syntactic Category	Valid AST	Invalid / Ambiguous
"Spends 20", "Decreased by 20"	Binary Operation	<code>SUB(State, 20)</code>	<code>ADD(State, -20)</code> (semantic pollution)
"A debt of 20", "20 below zero"	Terminal Literal	<code>-20</code>	<code>SUB(0, 20)</code> (unnecessary tree depth)
"Invert vector V"	Unary Operation	<code>NEG(V)</code>	<code>SUB(0, V)</code> (obfuscates operation arity)

4.6 Verification Checks

The system performs five automated verification checks before accepting any generated trace.

1. Arity Verification
2. Disambiguation Check
3. Dependency Check
4. Traceability Check
5. Operator Consistency Check

These checks ensure structural correctness, logical consistency, and trace validity.

5 System Architecture and Implementation

5.1 Forward Pipeline

The forward pipeline converts a natural language arithmetic problem into a verified multi-level intermediate representation.

The process consists of six stages.

5.1.1 Stage 1: LLM-Based Trace Generation

The arithmetic word problem is provided to a large language model along with:

- The EBNF grammar specification
- One-shot examples
- Output formatting constraints

The model generates reasoning trace lines without additional explanations or formatting.

5.1.2 Stage 2: Parsing Layer

The generated trace passes through a two-layer parser.

The first layer uses regular-expression tokenization to classify lines into:

- EXTRACT
- MAP
- BIND
- CONSTRUCT

The second layer uses recursive descent parsing to build Abstract Syntax Trees from prefix-notation expressions.

5.1.3 Stage 3: Schema Validation

The parsed traces are validated using strict Pydantic schemas.

The validation process ensures:

- Correct node types
- Proper operator arity
- Structural consistency
- Type correctness

5.1.4 Stage 4: Symbolic Execution

The generated AST is recursively executed to compute the final scalar result.

Each operation node is evaluated independently until the final answer is obtained.

5.1.5 Stage 5: Verification Checks

The framework performs the following verification procedures:

1. Arity verification
2. Disambiguation checking
3. Dependency validation
4. Traceability validation
5. Operator consistency verification

All checks must pass before the trace is accepted.

5.1.6 Stage 6: IR Conversion and Storage

The verified AST is transformed into:

- Raw AST
- Logic AST
- Canonical AST
- Abstract AST

SHA-256 hashes are generated for canonical representations and stored in a SQLite database.

5.2 Reverse Pipeline

The reverse pipeline generates new arithmetic word problems from abstract reasoning structures.

The generation occurs in three stages:

1. Convert Abstract AST into Canonical AST
2. Add contextual labels to generate Raw AST
3. Generate a natural language word problem

The generated problem is re-evaluated through the forward pipeline to verify structural equivalence.

5.3 Equivalence Simulator

The equivalence simulator validates whether structurally identical problems produce identical abstract representations.

Ten arithmetic problems with different wording but identical mathematical structures were processed independently through the pipeline.

All valid problems produced the same Abstract AST hash, demonstrating successful structural normalization.

5.4 Workspace Infrastructure

The system is deployed as multiple coordinated microservices.

Directory	Port	Purpose
gateway/	8000	Central routing hub
reasoning-grammar/	5001	Parsing and verification
parse-tree-as-ir/	8501	AST visualization and simulation

Table 1: System Microservices

The framework supports deployment using Docker Compose as well as native local execution.

6 Dataset Generation and Fine-Tuning Pipeline

The long-term objective of this framework is to generate high-quality structured reasoning traces for fine-tuning smaller language models capable of arithmetic reasoning without relying on large teacher models.

6.1 Structural Clustering and Operation Balancing

Arithmetic datasets often contain an imbalance of operations, particularly addition-heavy examples.

To address this issue, all problems are first converted into Abstract AST signatures and grouped according to:

- Structural depth
- Operator composition
- Reasoning patterns

Over-represented structures are capped, while under-represented structures are augmented through synthetic generation using the reverse pipeline.

6.2 Vocabulary Augmentation

To prevent lexical memorization, structurally identical problems are rewritten across multiple semantic domains.

For example, the same subtraction structure may appear as:

- Financial spending
- Temperature decrease
- Population decline
- Time reduction

This ensures that the model learns reasoning structures rather than memorizing keywords.

6.3 Training Schema

The training format follows a chat-completion style structure.

The user message contains the arithmetic word problem, while the assistant message contains the structured reasoning trace.

Example:

```
{
  "messages": [
    {
      "role": "user",
      "content": "Ravi has 5 apples and eats 2."
    },
    {
      "role": "assistant",
      "content": "[Extract] entity: '5 apples' -> val: 5"
    }
  ]
}
```

6.4 Curriculum Learning

The training process follows increasing structural complexity.

1. Depth-1 arithmetic operations
2. Depth-2 composite trees
3. Depth-3 nested structures
4. Fully shuffled balanced datasets

This curriculum allows the model to gradually learn syntax, operator precedence, and hierarchical reasoning.

6.5 Programmatic Verification

Evaluation is performed using deterministic symbolic execution instead of LLM-based judging.

The generated AST is parsed and executed programmatically.

If the computed result matches the expected ground truth answer, the reasoning trace is considered logically valid.

This evaluation strategy provides a reliable and reproducible accuracy metric independent of language-model grading variability.

7 Results and Observations

7.1 Model Size and JSON Extraction Reliability

A benchmarking study was conducted using models of different parameter sizes including:

- 3B
- 4B
- 7B
- 8B
- 20B
- 120B

The evaluation focused on three major criteria:

1. Structural fidelity
2. Arithmetic correctness
3. Surface diversity

The experiments revealed a clear capability threshold between 7B and 8B parameter models.

Models below 7B frequently generated:

- Structurally invalid parse trees
- Incorrect arithmetic operations
- Malformed JSON outputs

The 8B model demonstrated improved structural consistency but still struggled with strict schema compliance under complex prompting conditions.

Models with 20B parameters and above showed reliable performance across most evaluation scenarios.

The 120B model consistently generated structurally valid and semantically diverse outputs.

7.2 Equivalence Simulator Results

The equivalence simulator tested whether different arithmetic word problems sharing the same reasoning structure produced identical abstract representations.

Ten manually designed problems with different:

- Numerical values
- Surface wording
- Real-world contexts

were passed independently through the forward pipeline.

All valid problems generated identical Abstract AST hashes, demonstrating that the IR hierarchy successfully normalized:

- Operand ordering
- Numerical variation
- Linguistic differences

This validates the correctness of the canonicalization and abstraction process.

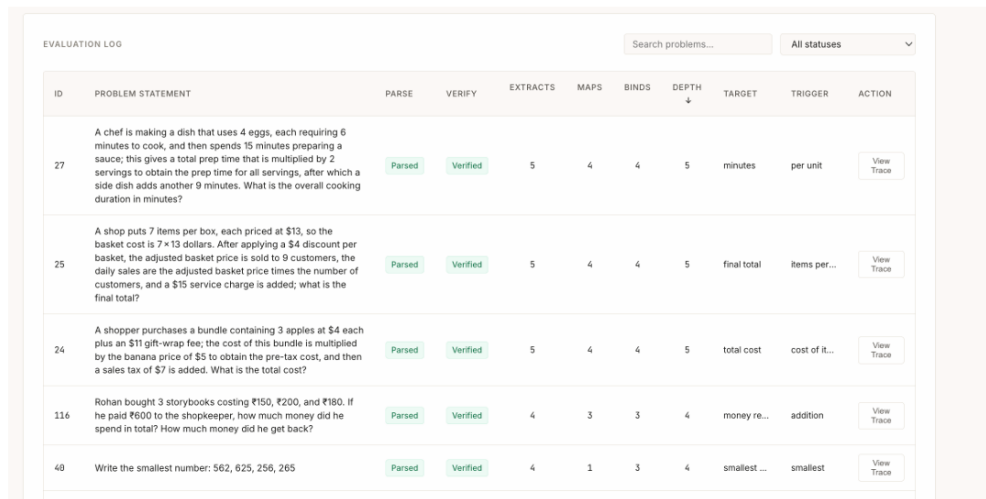
7.3 Batch Evaluation

A batch evaluation framework was developed to process arithmetic problems from the NCERT dataset.

The framework stores:

- Raw LLM outputs
- Parsed reasoning traces
- Computed scalar values
- Verification results
- Success and failure statistics

The evaluation infrastructure enables systematic analysis of pipeline reliability across different arithmetic operations and structural complexities.



The screenshot shows a web interface titled "EVALUATION LOG". It features a search bar labeled "Search problems..." and a dropdown menu labeled "All statuses". Below this is a table with the following columns: ID, PROBLEM STATEMENT, PARSE, VERIFY, EXTRACTS, MAPS, BINDS, DEPTH (with a downward arrow), TARGET, TRIGGER, and ACTION. The table contains five rows of data, each representing an arithmetic problem. The first two rows have "Parsed" and "Verified" status indicators. The last row has a "View Trace" button.

ID	PROBLEM STATEMENT	PARSE	VERIFY	EXTRACTS	MAPS	BINDS	DEPTH ↓	TARGET	TRIGGER	ACTION
27	A chef is making a dish that uses 4 eggs, each requiring 6 minutes to cook, and then spends 15 minutes preparing a sauce; this gives a total prep time that is multiplied by 2 servings to obtain the prep time for all servings, after which a side dish adds another 9 minutes. What is the overall cooking duration in minutes?	Parsed	Verified	5	4	4	5	minutes	per unit	View Trace
25	A shop puts 7 items per box, each priced at \$13, so the basket cost is 7×13 dollars. After applying a \$4 discount per basket, the adjusted basket price is sold to 9 customers, the daily sales are the adjusted basket price times the number of customers, and a \$15 service charge is added; what is the final total?	Parsed	Verified	5	4	4	5	final total	items per...	View Trace
24	A shopper purchases a bundle containing 3 apples at \$4 each plus an \$11 gift-wrap fee; the cost of this bundle is multiplied by the banana price of \$5 to obtain the pre-tax cost, and then a sales tax of \$7 is added. What is the total cost?	Parsed	Verified	5	4	4	5	total cost	cost of it...	View Trace
116	Rohan bought 3 storybooks costing ₹150, ₹200, and ₹180. If he paid ₹600 to the shopkeeper, how much money did he spend in total? How much money did he get back?	Parsed	Verified	4	3	3	4	money re...	addition	View Trace
40	Write the smallest number: 562, 625, 256, 265	Parsed	Verified	4	1	3	4	smallest ...	smallest	View Trace

8 Limitations and Future Work

8.1 Current Limitations

The current implementation supports a limited set of arithmetic operations including:

- ADD
- SUB
- MUL
- DIV

along with a small set of unary operations such as:

- NEG

- ABS
- SQRT
- LOG

The framework currently does not support:

- Algebraic manipulation
- Fractional reasoning
- Geometric theorem solving
- Symbolic algebra simplification

The canonicalization process handles commutativity for addition and multiplication but does not capture deeper algebraic equivalences such as distributive transformations. For example:

$$a \times (b + c)$$

and

$$a \times b + a \times c$$

are mathematically equivalent but currently generate different structural representations.

Another limitation is that Abstract AST structures capture reasoning shape but not problem difficulty. Two problems with very different complexity levels may still share the same abstract representation.

Smaller language models below 8B parameters frequently generate malformed reasoning traces and invalid JSON outputs, limiting the effectiveness of the current distillation pipeline for low-parameter models.

8.2 Future Work

The next major step is executing the full forward pipeline on the complete NCERT arithmetic dataset.

Future work includes:

1. Large-scale dataset generation
2. Fine-tuning small language models
3. Structural curriculum learning
4. Contrastive trace injection
5. Multilingual reasoning support

The framework will also be extended to support Hindi-language arithmetic reasoning in order to evaluate whether structured parse-tree representations transfer effectively across languages.

Another important direction is developing an edge-deployable educational assistant capable of:

- Generating verified reasoning traces
- Detecting student misconceptions
- Providing Socratic guidance
- Operating without internet connectivity

The long-term goal is to build a lightweight tutoring system powered by fine-tuned small language models capable of reliable and explainable arithmetic reasoning.

9 Conclusion

This project introduced a structured framework for grounding arithmetic reasoning in language models using dual grammars and multi-level parse tree representations.

The proposed architecture separates linguistic understanding from mathematical computation through:

- Reasoning Trace Grammar (GRT)
- Abstract Syntax Tree Grammar (GAST)
- Four-level IR hierarchy
- Verification-driven symbolic execution

The framework demonstrated the ability to:

- Verify arithmetic reasoning programmatically
- Detect structural inconsistencies
- Normalize equivalent reasoning structures
- Support synthetic problem generation

Experimental observations showed that larger language models produce significantly more reliable structured reasoning traces compared to smaller models.

The system establishes a foundation for future educational reasoning systems based on verifiable and interpretable arithmetic reasoning.

A LLM Prompt Templates

All prompts used in the system are centralised in `prompts.py` files within each subproject. The exact text sent to the LLM is reproduced below, since prompt engineering is a core part of the system’s correctness guarantees.

A.1 Reasoning Trace Generation Prompt

Source: reasoning-grammer/prompts.py → REASONING_TRACE_SYSTEM_PROMPT

Used in: reasoning-grammer/pipeline.py → process_problem()

```
You are a formal reasoning trace generator. Given a math word problem, you
must
produce a Reasoning Trace that follows this EXACT grammar:

## AST Grammar (prefix notation)

node                ::= binary_operation | unary_operation | terminal
binary_operation    ::= binary_operator [ metadata ] "(" node "," node ")"
unary_operation     ::= unary_operator [ metadata ] "(" node ")"
binary_operator     ::= "ADD" | "SUB" | "MUL" | "DIV" | "POW" | "MOD" | "MIN" | "
    MAX"
unary_operator      ::= "NEG" | "ABS" | "SQRT" | "LOG"
terminal            ::= literal | variable
literal             ::= [ "-" ] digits [ "." digits ]
variable            ::= identifier [ "[" annotation "]" ]
metadata            ::= "[" annotation "]"

## Reasoning Trace Grammar

reasoning_trace     ::= { trace_step } final_construction
trace_step          ::= step_id [ dependency_list ] step_body
step_id             ::= "[" identifier "]"
dependency_list     ::= "<" identifier { "," identifier } ">"
step_body           ::= extraction | mapping | binding

extraction          ::= EXTRACT entity: "..." -> val: terminal
mapping             ::= MAP trigger: "..." -> gen_trigger: "..." -> concept:
    "..." -> op: operator
binding            ::= BIND logic: "..." -> role: LEFT = node, RIGHT = node
    (or ARG = node for unary)
final_construction  ::= [ CONSTRUCT ] Tree: ast_expression -> target: "..."

## Disambiguation Rules

- A negative value that is an INTRINSIC PROPERTY (e.g. "a debt of 20")
  -> use negative literal: -20
- A RELATIONAL ACTION (e.g. "spends 20")
  -> use SUB(state, 20), NOT ADD(state, -20)
- INVERTING a variable (e.g. "opposite of X")
  -> use NEG(X), NOT SUB(0, X)

## Example

Problem: "An account has a debt of 20 dollars. A deposit of 100 dollars is
made.
Calculate the final balance."
```

```

Output:
[E1] EXTRACT entity: "debt of 20 dollars" -> val: -20
[E2] EXTRACT entity: "deposit of 100 dollars" -> val: 100
[M1] MAP trigger: "deposit is made" -> gen_trigger: "deposit"
    -> concept: "incremental addition to state" -> op: ADD
[B1] <E1, E2, M1> BIND logic: "Initial account state is modified by deposit
    amount"
    -> role: LEFT = -20, RIGHT = 100
[CONSTRUCT] Tree: ADD[balance](-20, 100) -> target: "dollars"

## Your Task
Output ONLY the reasoning trace lines. No explanations, no markdown fences,
no extra text.

```

A.2 Raw AST Extraction Prompt (Forward Pipeline)

Source: `parse-tree-as-ir/prompts.py` → `build_extraction_prompt(problem_text)`

You are a strict JSON math parser. Convert the following word problem into a single raw Abstract Syntax Tree (AST) root node.

Rules:

1. Output exactly ONE valid JSON object representing the root of the tree.
2. Output ONLY strictly valid JSON.
3. Trees must be STRICTLY BINARY.
4. Use operators:
 - add, sub, mul, div

Problem: {problem_text}

A.3 Reverse Pipeline Prompt — Abstract to Canonical

Source: `parse-tree-as-ir/prompts.py`

You are a math tree builder. You are given an ABSTRACT mathematical tree that shows ONLY the shape of operations with NO numbers.

Your job is to fill in concrete numeric values to turn this into a Canonical tree.

A.4 Reverse Pipeline Prompt — Canonical to Raw

Source: `parse-tree-as-ir/prompts.py`

You are a math story designer. You are given a Canonical math tree with operators and numbers.

Your job is to add a "label" field to each node to give it real-world meaning.

A.5 Reverse Pipeline Prompt — Raw to Word Problem

Source: `parse-tree-as-ir/prompts.py`

You are a math word problem writer. You are given a Raw AST tree that contains operators, numbers, and contextual labels.

Your job is to write ONE natural arithmetic word problem using the exact operations and numbers from the tree.

B References

1. Project Repository:
<https://github.com/mira-iitjmu/math-grounding-IR>
2. NCERT Mathematics Textbooks for Classes 3–5
3. Ollama Documentation:
<https://ollama.ai>
4. Pydantic Documentation:
<https://docs.pydantic.dev>
5. Graphviz Documentation:
<https://graphviz.org>